# Resumen Ejecutivo del Proyecto

- **Fecha de inicio:** Viernes 10/04/2026.
- **Repositorio:** Privado en GitHub bajo el nombre TicketAR-MundialUCP2026-.
- **Objetivo de la fase:** Inicialización técnica, configuración de seguridad y establecimiento del flujo de trabajo.

# 2. Configuración Técnica y Arquitectura

- **Blindaje de Seguridad (Rulesets):**
  - Restricción de borrado de la rama main.
  - Deshabilitación de push directo; uso obligatorio de Pull Requests (PR).
  - Revisión obligatoria con al menos una aprobación para integrar código.
  - Bloqueo de Force Push para proteger el historial.
- **Stack Tecnológico Inicial:**
  - **Frontend:** Next.js 14+ con HTML, Tailwind CSS y App Router.
  - **Backend:** Nest.js utilizando pnpm para optimizar dependencias.
  - **Documentación:** Carpeta /docs para diagramas UML y requerimientos.

# 3. Protocolo de Trabajo para el Equipo (Workflow)

Este es el ciclo de vida obligatorio para cualquier funcionalidad o cambio:

1. **Sincronización:** Antes de empezar, actualizar main localmente (git pull origin main).
2. **Branching:** Crear una rama lateral según la tarea (feat/, fix/, chore/).
3. **Desarrollo:** Programar en la rama específica, nunca en main.
4. **Commits:** Uso de prefijos estandarizados (Conventional Commits).
5. **Pull Request:** Apertura en GitHub para revisión del líder o compañeros.
6. **Merge & Clean:** Integración a main tras aprobación y borrado de la rama temporal.

## 📑 Parte 1: Comandos que corrimos hoy (Tu Setup)

| Comando | Para qué sirvió |
|---|---|
| `git init` | Le dijo a tu computadora: "Esta carpeta es un proyecto de Git, empezá a rastrear cambios". |
| `git remote add origin URL` | Creó el "puente" entre tu PC y el servidor de GitHub. |
| `git branch -M main` | Aseguró que tu rama principal se llame `main` (el estándar actual) y no `master`. |
| `git checkout -b nombre-rama` | Creó una **rama lateral**. Es como crear una "realidad paralela" para trabajar sin romper la versión oficial. |
| `npx create-next-app@latest` | Descargó e instaló toda la estructura profesional de **Next.js** (Frontend). |
| `npx @nestjs/cli new` | Descargó e instaló la estructura profesional de **Nest.js** (Backend). |
| `git add .` | Preparó todos los archivos nuevos para ser guardados (los puso en la "rampa de salida"). |
| `git commit -m "mensaje"` | Sacó una "foto" de los cambios y los guardó en el historial con una etiqueta descriptiva. |
| `git push origin nombre-rama` | Subió tus fotos (commits) desde tu PC a la nube de GitHub. |
| `git pull origin main` | Bajó los cambios de la nube a tu PC (lo usamos para sincronizar después del Merge). |

# Guía de Comandos y Setup del Desarrollador

Lista de comandos esenciales para que el equipo configure su entorno:

| Comando | Función |
| --- | --- |
| `npm install -g pnpm` | Instala el gestor de paquetes global necesario. |
| `git clone [URL]` | Descarga el repositorio al entorno local. |
| `pnpm install` | Instala dependencias en carpetas frontend y backend. |
| `git checkout -b [nombre]` | Crea y salta a una nueva rama de trabajo. |
| `git add .` | Prepara los cambios para el commit. |
| `git commit -m "[msj]"` | Guarda los cambios en el historial local. |
| `git push origin [rama]` | Sube la rama al servidor de GitHub. |

## Gestión de Escenarios Comunes

- **Sincronización en medio de una tarea:** Para traer cambios de un compañero sin perder el trabajo propio, se debe hacer commit del progreso actual, ir a main, hacer pull y luego hacer merge de main hacia la rama de tarea.
- **Inicio de nueva tarea:** Al terminar, se debe volver a main, actualizar con lo último del servidor y recién ahí crear la nueva rama.
- **Gestión de tiempos:** Las tareas están diseñadas para ser cortas y realizables día a día para asegurar el avance continuo del equipo.

### Escenario A: Estás en el medio de tu tarea y quieres lo que subió otro

Si Julian subió una función que vos necesitás usar ya mismo, hacés esto:

1. Guardás tus cambios (hacés git add . y git commit -m "work in progress"). Git no te deja traer cosas nuevas si tenés cambios "sueltos" sin guardar.
2. Vas a main: git checkout main.
3. Traés lo nuevo: git pull origin main.
4. Volvés a tu rama: git checkout feat/tu-tarea.
5. Unís lo nuevo a tu rama: git merge main.

**Resultado:** Tu código no se rompió, ni se borró. Y ahora se sumó lo que hizo Julian.

### Escenario B: Terminaste tu tarea y vas a empezar otra

Es el momento de "limpiar":

Vas a main: `git checkout main`.

Traés todo: `git pull origin main`.

Ahora que estás actualizado, creás la siguiente rama: `git checkout -b feat/nueva-tarea`.

Estos son escenarios que seguro pasen

Pero no quita los pasos lógicos o el flujo de trabajo que está mas arriba

## **Próximo paso:** Estructuración de GitHub Projects e Issues para el sábado 11/04/2026.